Lab 10 — Image Segmentation and Feature Extraction

Course: ARTI404 – Image Processing
College of Computer Science and Information Technology
Imam Abdulrahman Bin Faisal University

This notebook contains:

1. The two examples from the lab manual (Canny Edge Detection, Harris Corner Detection).
2. The assessment task (Otsu's thresholding).

Setup — Import libraries

In [1]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import data   # provides sample images (camera, checkerboard, coins

# Make plots show inside the notebook
%matplotlib inline
```

Task 1 — Canny Edge Detection

An **edge** is where brightness changes sharply. Canny (1986) finds edges in 5 steps: Gaussian blur → gradients → non-maximum suppression (thin the edges) → double thresholding → hysteresis (keep weak edges only if they connect to strong ones).

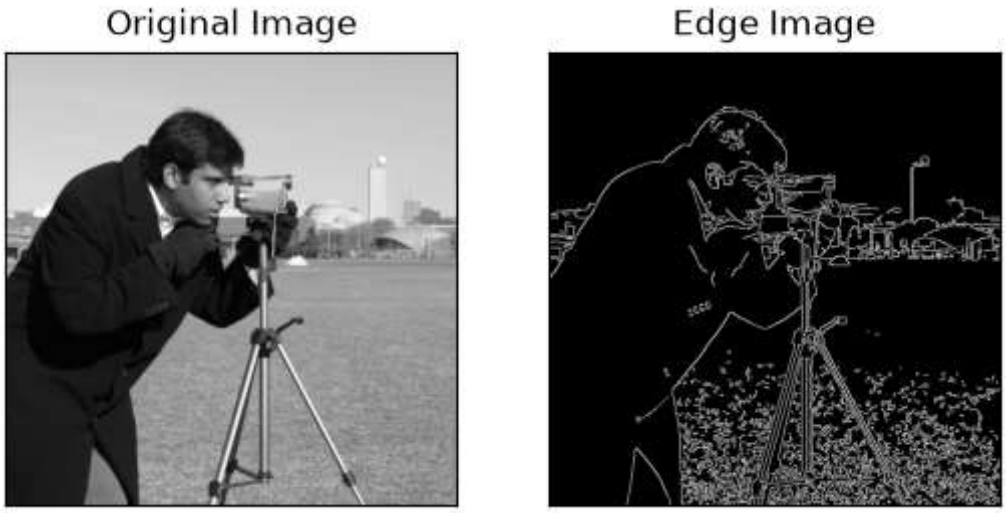
`cv2.Canny(img, T_low, T_high)` :

- `T_low` = lower threshold (weak edges)
- `T_high` = upper threshold (strong edges)
- Rule of thumb: `T_high ≈ 2×T_low` or `3×T_low`.

In [2]:
```
# --- Code from the lab manual ---
img = data.camera()

edges = cv2.Canny(img, 100, 200)

plt.subplot(121), plt.imshow(img, cmap='gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122), plt.imshow(edges, cmap='gray')
plt.title('Edge Image'), plt.xticks([]), plt.yticks([])
plt.show()
```



Task 2 — Harris Corner Detection

A **corner** is a point where the image changes in *two* different directions — which makes corners highly distinctive (useful for matching, tracking, camera calibration).

- Large positive `R` → corner
- Large negative `R` → edge
- Small `R` → flat region

Parameters: `blockSize` (window size), `ksize` (Sobel kernel size), `k` (sensitivity, typically 0.04–0.06).

> **Note:** OpenCV uses BGR order, so `(0, 0, 255)` is red.

In [3]:
```
# --- Code from the lab manual ---
board = data.checkerboard()
image = cv2.cvtColor(board, cv2.COLOR_GRAY2BGR)
image = cv2.resize(image, (400, 400), interpolation=cv2.INTER_NEAREST)

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

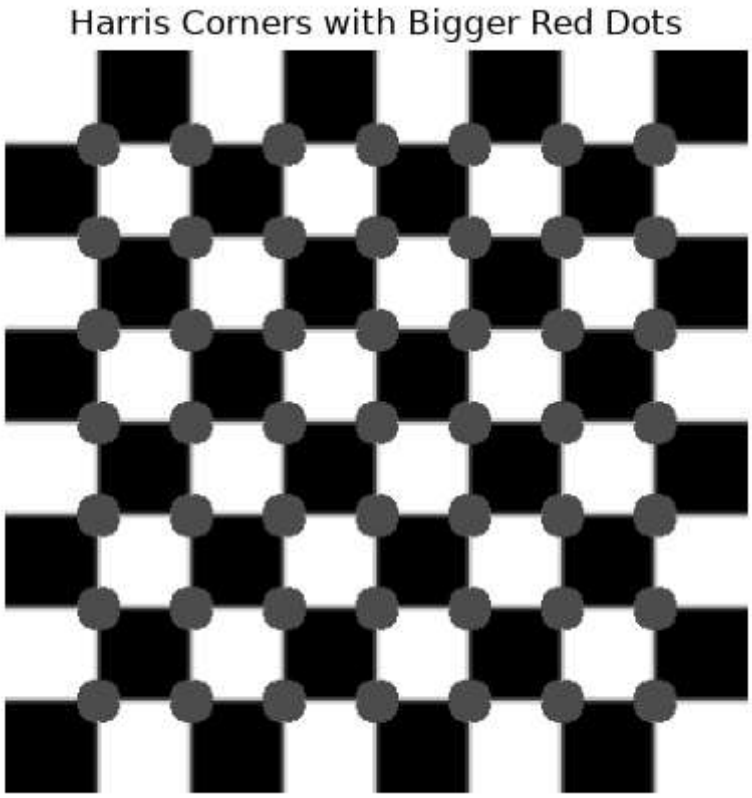
# Harris Corner Detection
harris = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
harris = cv2.dilate(harris, None)   # to mark corners more clearly

# Threshold for detecting strong corners
threshold = 0.01 * harris.max()
corner_coords = np.argwhere(harris > threshold)

# Draw larger red dots (radius = 7)
for y, x in corner_coords:
    cv2.circle(image, (x, y), radius=7, color=(0, 0, 255), thickness=-1)  # -1 =

# Display
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Harris Corners with Bigger Red Dots')
```

```
plt.axis('off')
plt.show()
```



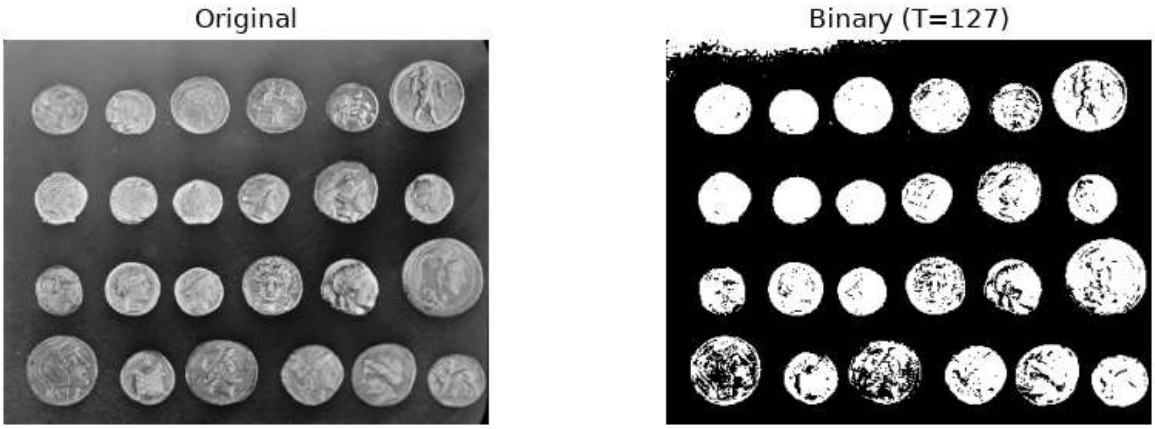
Simple (Global) Binary Thresholding

The most basic segmentation: pick a fixed threshold `T` and turn every pixel above it white, below it black.

In [4]:
```
img = data.coins()

T = 127
_, binary = cv2.threshold(img, T, 255, cv2.THRESH_BINARY)

plt.figure(figsize=(9, 3))
plt.subplot(1, 2, 1), plt.imshow(img, cmap='gray'), plt.title('Original'), plt.a
plt.subplot(1, 2, 2), plt.imshow(binary, cmap='gray'), plt.title(f'Binary (T={T}
plt.tight_layout(), plt.show()
```



Out[4]: (None, None)

Q1 — Otsu's Thresholding (Assessment Task)

Otsu picks `T` **automatically** by maximizing the between-class variance — assumes the histogram has two peaks (foreground + background).

show the original image, its histogram, and the thresholded image.

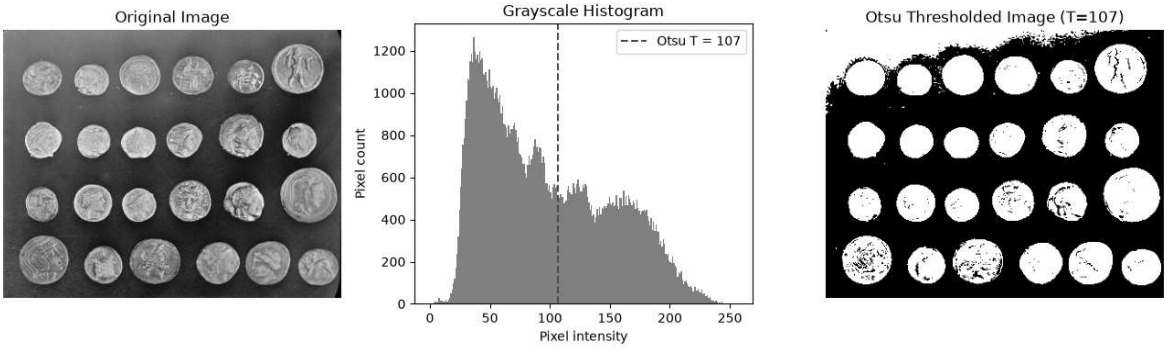
In [5]:
```
# Q1: Otsu's automatic thresholding (using the coins image)
img_otsu = data.coins()

# Otsu chooses the threshold automatically (the supplied threshold is ignored).
otsu_T, otsu_binary = cv2.threshold(
    img_otsu, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
axes[0].imshow(img_otsu, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].hist(img_otsu.ravel(), bins=256, range=(0, 256), color='gray')
axes[1].axvline(otsu_T, color='red', linestyle='--', label=f'Otsu T = {otsu_T:.0
axes[1].set_title('Grayscale Histogram')
axes[1].set_xlabel('Pixel intensity')
axes[1].set_ylabel('Pixel count')
axes[1].legend()

axes[2].imshow(otsu_binary, cmap='gray', vmin=0, vmax=255)
axes[2].set_title(f'Otsu Thresholded Image (T={otsu_T:.0f})')
axes[2].axis('off')
plt.tight_layout()
plt.show()
print(f'Automatically selected Otsu threshold: {otsu_T:.0f}')
```




```
Automatically selected Otsu threshold: 107
```

Observation: Otsu's method automatically selects a grayscale threshold from the image histogram. Pixels above the selected threshold become white, while pixels at or below it become black. Unlike global thresholding with a manually chosen value, Otsu uses the image intensity distribution to determine the threshold.

Summary table

Technique	Category	What it finds	OpenCV function
Canny	Edge detection	Thin connected edges	`cv2.Canny`
Sobel	Edge detection	Gradient magnitude (thick)	`cv2.Sobel`
Laplacian	Edge detection	Zero-crossings (2nd deriv)	`cv2.Laplacian`
Harris	Feature extraction	Corners	`cv2.cornerHarris`
Shi-Tomasi	Feature extraction	Corners (improved)	`cv2.goodFeaturesToTrack`
Simple threshold	Segmentation	Fixed-`T` binary mask	`cv2.threshold`
Otsu	Segmentation	Auto bimodal threshold	`cv2.threshold` + `THRESH_OTSU`
Adaptive threshold	Segmentation	Local-`T` binary mask	`cv2.adaptiveThreshold`

End of notebook.